

# OPC UA Mock Server 用户手册

本文档说明如何部署与运行 OPC UA Mock Server，以及如何编写组态文件。

## 一、如何使用

打包后，可执行文件在 `dist/` 目录下。运行方式：

```
./ua_mockер config_example.yaml
```

(Windows 下可为 `ua_mockер.exe config_example.yaml`。)

### 1.4 启动后控制台输出

控制台仅会输出以下信息（无其他日志）：

- 开始构建节点树
- 构建完成
- 服务启动成功 `opc.tcp://<地址>:<端口>/ua_mockер/`
- 节点数量：<数量>
- designed by yzc
- 当有客户端对可写节点写值时，会多一行：写值 `NodeId=... Value=...`

其余日志（组态加载、错误等）只写入日志文件，不输出到控制台。

### 1.5 客户端连接

- 端点地址：** `opc.tcp://<server>:<port>/ua_mockер/`  
例如组态中 `server: "0.0.0.0"`、`port: 18950` 时，为  
`opc.tcp://0.0.0.0:18950/ua_mockер/`；本机连接可用  
`opc.tcp://127.0.0.1:18950/ua_mockер/`。
- NodeId 形式：** `ns=<namespace_index>;s=<节点名>`  
例如组态中 `namespace_index: 1`、节点 `name: "node_float_"` 且 `count: 3` 时，三个节点为：`ns=1;s=node_float_1`、`ns=1;s=node_float_2`、`ns=1;s=node_float_3`。
- 读：**所有节点均可读。
- 写：**仅当该节点在组态中 `writable: true` 时可写；写成功后值会持久化，下次读会得到写入值。

## 1.6 日志文件

- **位置：**执行程序所在目录（脚本方式为 `ua_mock` 目录，打包后为 `exe` 所在目录）。
- **文件名：** `ua_mock_YYYYMMDD.log`（按天）。
- **内容：**组态加载、端点、命名空间、服务器启动、客户端写值等完整日志。

## 1.7 注意事项

- 组态文件不存在或格式错误时，程序会报错并退出。
- 端口被占用时启动会失败，请修改组态中的 `port` 或释放端口。
- 当前未实现用户权限与安全策略，仅支持匿名访问。

# 二、如何写组态文件

组态文件为 `YAML` 格式，用于配置服务地址、端口、数据变化周期、命名空间和节点列表。

## 2.1 文件结构概览

顶层必须包含以下键：

| 键名                           | 类型  | 必填 | 说明                                                           |
|------------------------------|-----|----|--------------------------------------------------------------|
| <code>server</code>          | 字符串 | 是  | 监听地址，如 <code>"0.0.0.0"</code> 表示本机所有网卡                       |
| <code>port</code>            | 整数  | 是  | 端口号，如 <code>18950</code>                                     |
| <code>cycle</code>           | 整数  | 是  | 数据自动变化周期（毫秒），如 <code>1000</code> 表示 1 秒                      |
| <code>namespace_index</code> | 整数  | 是  | OPC UA 命名空间索引，如 <code>1</code> （客户端看到的 <code>ns</code> 与此一致） |
| <code>nodes</code>           | 列表  | 是  | 节点定义列表，见下文                                                   |

## 2.2 顶层配置示例

```
server: "0.0.0.0"
port: 18950
cycle: 1000
namespace_index: 1

nodes:
  # 下面填写节点列表
```

## 2.3 节点项 (nodes 中每一项)

每个节点项是一个对象，包含以下字段：

| 字段       | 类型    | 必填   | 说明                                                            |
|----------|-------|------|---------------------------------------------------------------|
| name     | 字符串   | 是    | 名称前缀。与下标拼接得到节点名，如 "node_float_" + 下标 1 → node_float_1         |
| type     | 字符串   | 是    | OPC UA 数据类型，见下表                                               |
| count    | 整数    | 是    | 该 name 下生成的节点个数，节点名为 name_1、name_2 ... name_count             |
| default  | 视类型而定 | 条件必填 | 初始值。 <b>仅当 change 为 false 时必填</b> ；change 为 true 时不要写 default |
| change   | 布尔    | 是    | 是否按 cycle 周期自动变化                                              |
| writable | 布尔    | 是    | 是否允许客户端写；为 true 时写成功后会持久化                                     |

## 2.4 支持的数据类型 (type)

| type 取值  | 说明         | default 示例 (change=false 时)        |
|----------|------------|------------------------------------|
| Boolean  | 布尔         | true / false                       |
| SByte    | 有符号 8 位整数  | 0                                  |
| Byte     | 无符号 8 位整数  | 0                                  |
| Int16    | 有符号 16 位整数 | 0                                  |
| UInt16   | 无符号 16 位整数 | 0                                  |
| Int32    | 有符号 32 位整数 | 0                                  |
| UInt32   | 无符号 32 位整数 | 0                                  |
| Int64    | 有符号 64 位整数 | 0                                  |
| UInt64   | 无符号 64 位整数 | 0                                  |
| Float    | 32 位浮点     | 0.0                                |
| Double   | 64 位浮点     | 0.0                                |
| String   | 字符串        | "" 或任意字符串                          |
| DateTime | 日期时间       | "2025-01-01T00:00:00Z" 或 ISO 格式字符串 |

## 2.5 change 为 true 时的自动变化规则

当节点 `change: true` 时，**不需要写** `default`，值由程序按周期自动更新：

- **周期**：使用顶层配置的 `cycle`（毫秒）。
- **Boolean**：每个周期在 true / false 之间翻转。
- **数值类型**（如 Int32、Float 等）：0~99 的锯齿波，步长 1，到 99 后回到 0。
- **String**：单字符在 a~z 之间循环，每周期切到下一个字母。
- **DateTime**：每周期在时间上增加 1 秒。

## 2.6 节点命名与 NodeId

- 节点名 (BrowseName / DisplayName) = 配置中的 `name` + 下标。  
例如 `name: "node_float_"`、`count: 3` → `node_float_1`、`node_float_2`、`node_float_3`。
- NodeId 形式：`ns=<namespace_index>;s=<节点名>`。  
例如 `namespace_index: 1` 时，上述三个节点为：`ns=1;s=node_float_1`、`ns=1;s=node_float_2`、`ns=1;s=node_float_3`。

## 2.7 组态示例

### 示例 1: 3 个不变化、可写的 Float 节点

```
server: "0.0.0.0"
port: 18950
cycle: 1000
namespace_index: 1

nodes:
  - name: "node_float_"
    type: Float
    count: 3
    default: 0.0
    change: false
    writable: true
```

得到节点: `ns=1;s=node_float_1`、`ns=1;s=node_float_2`、`ns=1;s=node_float_3`，初始值 0.0，客户端可写。

### 示例 2: 2 个自动变化、只读的 Int32 节点

```
server: "0.0.0.0"
port: 18950
cycle: 1000
namespace_index: 1

nodes:
  - name: "node_int32_"
    type: Int32
    count: 2
    change: true
    writable: false
```

得到节点: `ns=1;s=node_int32_1`、`ns=1;s=node_int32_2`，值按 0~99 锯齿波、每 1 秒变化，不可写。

### 示例 3: 混合使用（部分自变化只读 + 部分固定可写）

```
server: "0.0.0.0"
port: 18950
cycle: 1000
namespace_index: 1

nodes:
  - name: "bool_ch_"
    type: Boolean
    count: 2
    change: true
    writable: false
  - name: "bool_wr_"
    type: Boolean
    count: 2
    default: false
    change: false
    writable: true
```

- `bool_ch_1`、`bool_ch_2`：每 1 秒 true/false 翻转，只读。
- `bool_wr_1`、`bool_wr_2`：初始值 false，客户端可写，写后持久化。

## 2.8 完整示例文件

项目根目录下的 `config_example.yaml` 为完整示例，包含所有支持类型，且每种类型都包含「change=true 只读」与「change=false 可写」的节点，可直接用作模板或测试。

---

## 三、常见问题

- **组态报错“缺少 default”**：该节点为 `change: false`，必须为该节点填写 `default`。
- **客户端连不上**：检查防火墙、`server / port` 是否与客户端一致；本机测试可用 `127.0.0.1`。
- **不知道 NodeId**：按 `ns=<namespace_index>;s=<name><下标>` 拼接，或在客户端中浏览 Objects 下的节点查看。